

THE COLLEGE

B.Tech Final Year Project Report

Title: The College - Full-Stack Notes Management and Student Tools Platform

Submitted by: [STUDENT NAME] [ROLL NO] [BRANCH] [SEMESTER]

Under the guidance of: [GUIDE NAME] [DEPARTMENT]

[COLLEGE NAME] [UNIVERSITY NAME] [SESSION YEAR]

CERTIFICATE

This is to certify that the project entitled "The College - Full-Stack Notes Management and Student Tools Platform" is a bonafide record of the work carried out by [STUDENT NAME] (Roll No: [ROLL NO]) under my supervision and guidance, in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in [BRANCH] during the academic session [SESSION YEAR] at [COLLEGE NAME].

Guide: [GUIDE NAME] [DESIGNATION] [DEPARTMENT]

Date: [DATE] Place: [PLACE]

CANDIDATE DECLARATION

I, [STUDENT NAME] (Roll No: [ROLL NO]), hereby declare that the work presented in this B.Tech Final Year Project Report titled "The College - Full-Stack Notes Management and Student Tools Platform" is an original record of my work carried out under the supervision of [GUIDE NAME]. I confirm that this report has not been submitted in part or full to any other university or institution for the award of any degree or diploma.

Date: [DATE] Place: [PLACE]

Signature of Candidate

ACKNOWLEDGEMENT

I express my sincere gratitude to [GUIDE NAME] for valuable guidance, continuous encouragement, and constructive feedback throughout the project. I also thank the faculty of [DEPARTMENT] and [COLLEGE NAME] for providing the infrastructure and academic environment that enabled the successful completion of this work. I am grateful to my peers and family for their support and motivation.

CONTENTS

1. Title Page
2. Certificate
3. Candidate Declaration
4. Acknowledgement
5. Contents
6. CHAPTER I: INTRODUCTION
 - 1.1 Abstract / Project Synopsis
 - 1.2 Background and Problem Statement
 - 1.3 Scope and Objectives of Present Work
 - Table 1.1: Existing vs Proposed System
7. CHAPTER II: SYSTEM ANALYSIS AND REQUIREMENT SPECIFICATIONS
 - 2.1 Software Requirement Specification (SRS)
 - 2.1.1 Functional and Non-Functional Requirements
 - 2.1.1.1 Hardware Interfaces and Requirements
 - 2.1.1.2 Software Interfaces, Stack and Tools Used
 - Table 2.2: Database Table Schema
8. CHAPTER III: SYSTEM DESIGN AND SYSTEM ARCHITECTURE
 - DFD Level 0
 - DFD Level 1
 - ER Diagram
 - System Flowchart
 - Database Schema / Table Design
 - Figure Index entries
9. CHAPTER IV: IMPLEMENTATION AND WORKING PRINCIPLE
 - Core Algorithms
 - Module Descriptions
 - Key Code Snippets
 - Frontend/Backend Connectivity
10. CHAPTER V: TESTING, RESULTS AND APPLICATION
 - Test Case Tables
 - UI Screenshot Placeholders
 - Output Analysis
 - Real-world Applications
11. CHAPTER VI: ADVANTAGES, LIMITATIONS AND FUTURE SCOPE
12. REFERENCES
13. TABLE INDEX
14. FIGURE INDEX
15. APPENDICES

CHAPTER I: INTRODUCTION

1.1 Abstract / Project Synopsis

The College is a full-stack educational platform that centralizes academic notes and learning tools for students, administrators, and super administrators. The system provides a structured academic hierarchy (session -> course -> branch -> semester -> subject -> notes), secure role-based authentication, and a modern web interface built with React and Vite. The backend is implemented with Node.js and Express, uses MongoDB for persistent data, integrates Cloudinary for note storage, and provides a PostgreSQL query sandbox for SQL practice. The platform also includes an AI assistant module, a feedback system, contact management, and a newsletter subscription feature. The application name "The College" is used throughout the repository, while the client HTML title and assistant prompt also reference "Hellomates / The College" in the UI and chatbot context.

1.2 Background and Problem Statement

Academic notes and resources are often shared through informal channels such as personal messaging apps, email threads, and scattered files, which creates problems in discoverability, quality control, and long-term accessibility. Institutions also require role-specific oversight, such as administrative approval of materials and system-wide academic structure management. The College addresses these issues by providing a secure, centralized platform with hierarchical organization, verified uploads, feedback loops, and additional learning tools.

1.3 Scope and Objectives of Present Work

Scope:

- Provide a centralized, role-based platform for academic notes.
- Offer learning tools such as compilers and practice labs through a unified interface.
- Support both traditional user authentication and social login via Firebase.
- Provide administrative control for academic hierarchy management and user oversight.

Objectives:

- Implement secure authentication with JWT and httpOnly cookies.
- Organize academic content using a structured hierarchy of session, course, branch, semester, subject, and notes.
- Implement a robust search and filtering system for notes.
- Provide a backend API for notes, feedback, contact, subscription, academic structure, and compilers.
- Provide an interactive frontend for students, admins, and super admins.

Table 1.1: Existing vs Proposed System

Aspect	Existing System	Proposed System (The College)
Notes Access	Scattered files, manual sharing	Centralized, role-based repository
Organization	Ad-hoc or folder-based	Structured hierarchy (session -> course -> branch -> semester -> subject -> notes)
Authentication	Often informal	JWT-based, email verification, httpOnly cookies
Admin Control	Limited or manual	Admin and super admin controls in API and UI

Aspect	Existing System	Proposed System (The College)
Search	Manual discovery	Server-side text search + filter parameters
Learning Tools	Separate websites or none	Integrated tools hub with compilers and labs
Feedback	Informal	Feedback API and admin review

CHAPTER II: SYSTEM ANALYSIS AND REQUIREMENT SPECIFICATIONS

2.1 Software Requirement Specification (SRS)

The system follows a modular client-server architecture. The backend provides REST endpoints for authentication, academic structure, notes, feedback, contact, subscription, chatbot, and compilers. The frontend provides protected routes for users, admins, and super admins with a consistent UI and contextual data loading.

2.1.1 Functional and Non-Functional Requirements

Functional Requirements

ID	Requirement	Description
FR-01	User registration	Users can register with name, email, password, and academic metadata.
FR-02	Email verification	Users, admins, and super admins verify email via JWT links.
FR-03	User login	Verified users can login and receive JWT via cookie.
FR-04	Admin login	Admins login and manage notes within assigned course/department.
FR-05	SuperAdmin login	SuperAdmin manages academic structure and users/admins.
FR-06	Notes upload	Admins upload notes with file and metadata; Cloudinary stores files.
FR-07	Notes search	Users search with text and filters (session, course, branch, semester, subject).
FR-08	Notes listing	Authenticated users view notes; public home data shows summary.
FR-09	Feedback system	Users submit feedback; admin/superadmin can view and delete.

ID	Requirement	Description
FR-10	Contact system	Users submit contact forms; admins view and delete.
FR-11	Subscription	Users subscribe/unsubscribe to newsletter; admin can manage list.
FR-12	Academic CRUD	SuperAdmin creates and manages sessions, courses, branches, semesters, subjects.
FR-13	Chatbot assistant	Authenticated users send messages to AI assistant endpoint.
FR-14	Compilers	C, C++, Java, Python compilers; PostgreSQL query sandbox.
FR-15	Tool hub	Tools catalog provides navigation to compilers and learning modules.

Non-Functional Requirements

ID	Requirement	Description
NFR-01	Security	JWT, bcrypt password hashing, role-based access, httpOnly cookies.
NFR-02	Performance	Text index for notes search; pagination parameters in API.
NFR-03	Availability	Dockerized deployment profiles for dev and prod.
NFR-04	Maintainability	Modular structure with controllers, routes, models, and components.
NFR-05	Portability	Works in Docker and local environment; Vite proxy supports dev.
NFR-06	External dependency handling	Cloudinary, Firebase, and chatbot provider configured via env.

2.1.1.1 Hardware Interfaces and Requirements

Category	Requirement
CPU	[TBD] (standard development machine capable of running Node.js and compilers)
RAM	[TBD]
Storage	[TBD] (local storage for dependencies and build artifacts)
Network	Internet connectivity for Cloudinary, Firebase, and chatbot providers

Category	Requirement
Compiler toolchain	gcc, g++, javac/java, and python or py -3 available on server host

2.1.1.2 Software Interfaces, Stack and Tools Used

Layer	Tools/Interfaces (from repository)	Version / Notes
Frontend	React, React Router, Vite, Tailwind CSS	React 19, React Router 7, Vite 6, Tailwind 4
Backend	Node.js, Express	Express 4.21.2
Database	MongoDB, Mongoose	Mongoose 8.10.0
SQL Sandbox	PostgreSQL, pg client	pg 8.20.0
File Storage	Cloudinary, multer-storage-cloudinary	Cloudinary 2.5.1
Auth	JWT, bcryptjs, cookie-parser	jsonwebtoken 9.0.2, bcryptjs 2.4.3
Email	Nodemailer	nodemailer 6.10.0
Social Auth	Firebase client + Firebase Admin	firebase 12.10.0, firebase-admin 13.7.0
AI Assistant	Gemini or OpenAI-compatible API	Configured via env
DevOps	Docker Compose, Nginx	dev and prod profiles

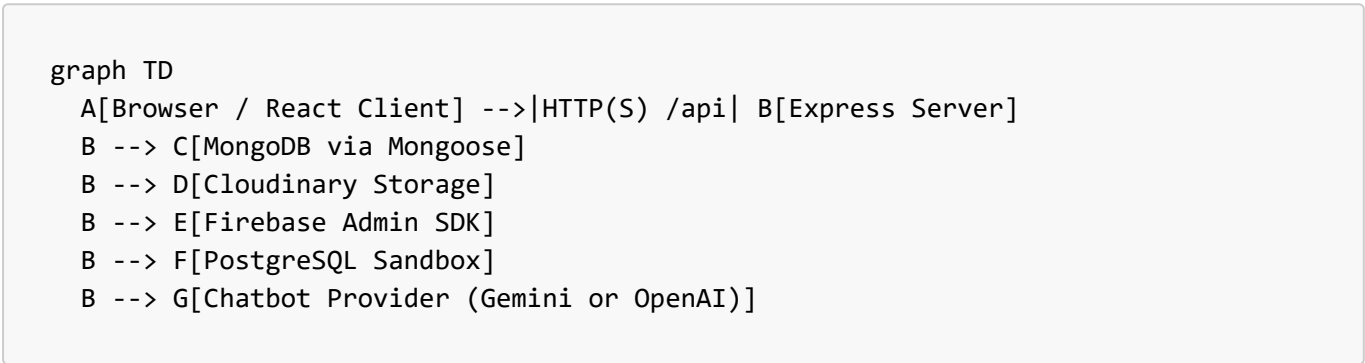
Table 2.2: Database Table Schema (MongoDB Collections)

Collection	Key Fields (summary)
User	name, course, branch, enrollment, email, password, profilePic, isVerified, role, authProvider, firebaseUid, timestamps
Admin	name, email, password, role, isVerified, course, department, college, designation, timestamps
SuperAdmin	name, email, password, role, isVerified, timestamps
Note	title, description, fileUrl, cloudinaryId, uploadedBy (refPath), uploaderModel, session, course, branch, semester, subject, timestamps
Session	year, startYear, endYear, isActive, createdBy, timestamps
Course	name, description, code, icon, color, isActive, createdBy, timestamps
Branch	name, fullName, code, course (ref), isActive, createdBy, timestamps
Semester	name, number, course (ref), isActive, createdBy, timestamps
Subject	name, code, branch (ref), semester (ref), course (ref), isActive, createdBy, timestamps
Feedback	user (ref), message, rating, timestamps
Contact	user (ref), name, email, phone, message

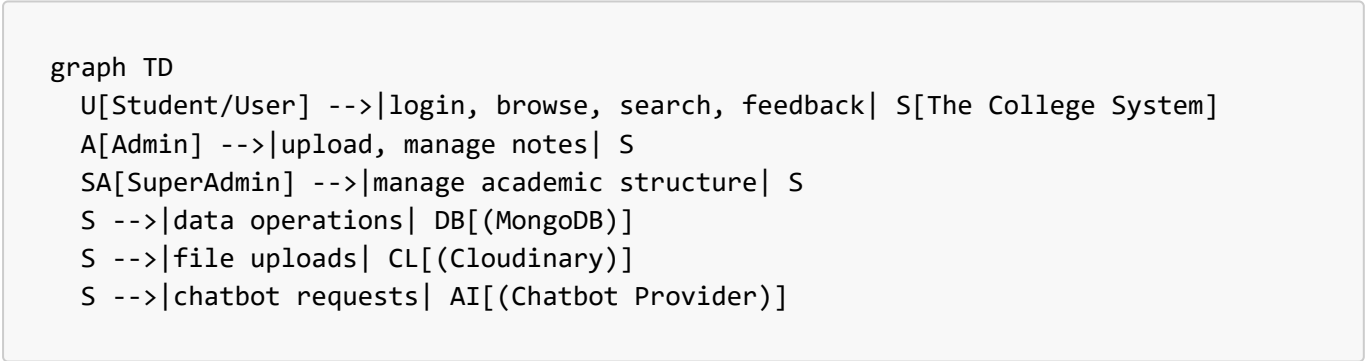
Collection	Key Fields (summary)
Subscribe	email, timestamps
Event	title, type, date, startTime, endTime, location, description, image, organizer, registrationUrl, isActive, timestamps

CHAPTER III: SYSTEM DESIGN AND SYSTEM ARCHITECTURE

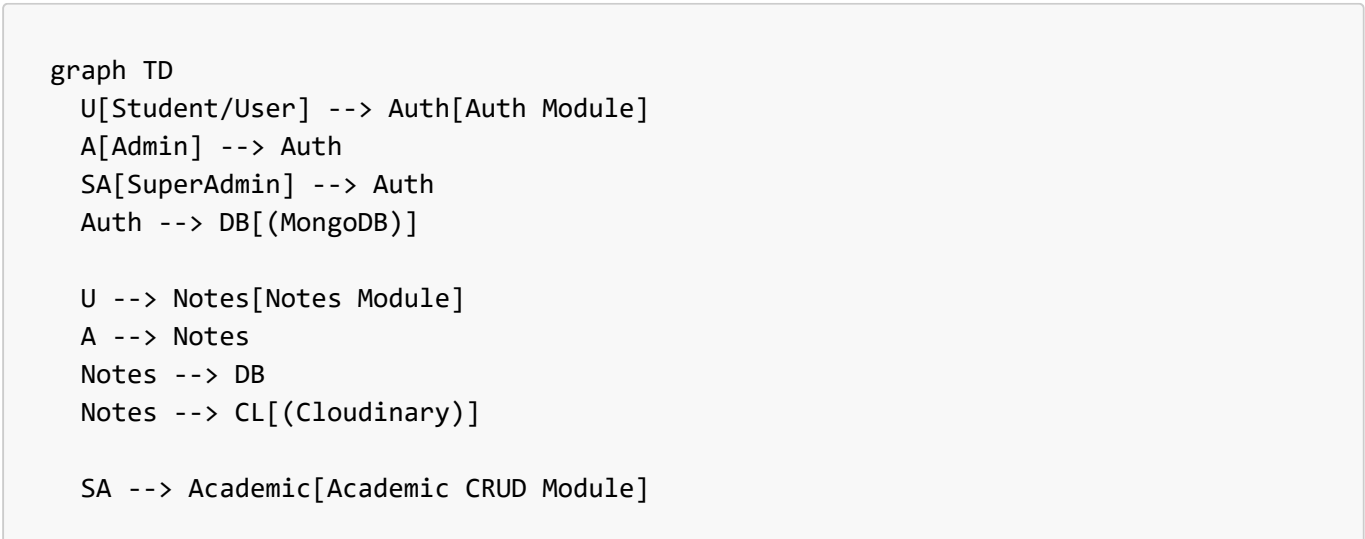
Overall Architecture



DFD Level 0



DFD Level 1



Academic --> DB

U --> Feedback[Feedback and Contact]
Feedback --> DB

U --> Tools[Compilers and Tools]
Tools --> PG[(PostgreSQL Sandbox)]

U --> Assistant[AI Assistant]
Assistant --> AI[(Chatbot Provider)]

ER Diagram (conceptual)

```
erDiagram
    SUPERADMIN ||--o{ SESSION : creates
    SUPERADMIN ||--o{ COURSE : creates
    SUPERADMIN ||--o{ BRANCH : creates
    SUPERADMIN ||--o{ SEMESTER : creates
    SUPERADMIN ||--o{ SUBJECT : creates
    COURSE ||--o{ BRANCH : has
    COURSE ||--o{ SEMESTER : has
    BRANCH ||--o{ SUBJECT : has
    SEMESTER ||--o{ SUBJECT : has
    USER ||--o{ NOTE : uploads
    ADMIN ||--o{ NOTE : uploads
    USER ||--o{ FEEDBACK : submits
    USER ||--o{ CONTACT : submits
```

System Flowchart

```
flowchart TD
    Start([Start]) --> Role{Select Role}
    Role -->|Student| SLogin[User Login]
    Role -->|Admin| ALogin[Admin Login]
    Role -->|SuperAdmin| SLogin[SuperAdmin Login]

    SLogin --> SHome[Dashboard -> Courses -> Branches -> Semesters -> Subjects]
    SHome --> NotesList[View and Search Notes]
    NotesList --> Feedback[Submit Feedback]
    NotesList --> Assistant[Chatbot Assistant]

    ALogin --> APanel[Admin Dashboard]
    APanel --> Upload[Upload Notes]
    APanel --> Manage[Manage Notes]

    SLogin --> SPanel[SuperAdmin Dashboard]
    SPanel --> AcademicCRUD[Sessions/Courses/Branches/Semesters/Subjects]
    SPanel --> UserAdminMgmt[Manage Users/Admins]
```



```
NotesList --> End([End])
```

Database Schema / Table Design (summary)

- The MongoDB schema follows separate collections for user roles, academic hierarchy, notes, and support entities.
- The Note collection uses a dynamic reference (refPath) to support uploads by users or admins.
- The Session, Course, Branch, Semester, and Subject collections form the academic hierarchy.

Figure Index Entries (for Chapter III)

- Figure 3.1: Overall System Architecture
- Figure 3.2: DFD Level 0
- Figure 3.3: DFD Level 1
- Figure 3.4: ER Diagram
- Figure 3.5: System Flowchart

CHAPTER IV: IMPLEMENTATION AND WORKING PRINCIPLE

Core Algorithms

1. Authentication with JWT cookies

- Tokens are generated during login and stored as httpOnly cookies.
- Requests validate tokens via middleware with fallback to Authorization header.

2. Note upload validation

- Admin uploads are validated against assigned course and branch.
- Subject and semester are verified against active academic data.
- Cloudinary ID is captured and stored for deletion on update or delete.

3. Notes search and filtering

- Uses MongoDB text index for title, description, subject.
- Additional filters for session, course, branch, semester, subject.
- Supports pagination and relevance sorting when text search is used.

4. Compiler execution

- C/C++/Java/Python code is stored in temporary files.
- Code is compiled (where applicable), executed with timeouts, and cleaned up.

5. PostgreSQL sandbox

- Creates temporary tables, runs the query in a transaction, and rolls back.

- Provides both direct query endpoint and sandbox preview endpoint.

Module Descriptions (high level)

Backend Modules:

- Auth module: registration, login, email verification, profile updates.
- Academic module: CRUD for sessions, courses, branches, semesters, subjects.
- Notes module: upload, update, delete, list, search, public home data.
- Feedback/Contact/Subscribe module: user communication and admin review.
- Assistant module: chatbot interface with provider selection.
- Compiler module: C, C++, Java, Python compilers and PostgreSQL sandbox.

Frontend Modules:

- AuthContext: manages user, admin, and super admin sessions.
- Protected routes: guard access based on role.
- Notes UI: list, filter, and display notes with metadata.
- Tools Hub: lists compilers and learning tools with navigation paths.
- Home page: shows latest notes, stats, and testimonials.

Documentation Modules:

- Docs in the repository describe tool designs such as API tester, JSON formatter, regex tester, DSA visualizer, and more.

Key Code Snippets (from repository)

Snippet A: Express app setup and routes

```
// Server/server.js
app.use("/api/auth", authRoutes);
app.use("/api/notes", noteRoutes);
app.use("/api/feedback", feedbackRoutes);
app.use("/api/contact", ContactRoutes);
app.use("/api/superadmin", superAdminRoutes);
app.use("/api/subscribe", subscribeRoutes);
app.use("/api/academic", academicRoutes);
app.use("/api/chatbot", chatbotRoutes);
app.use("/api/query", postgresCompilerRoutes);
app.use("/api/compile/", cCompilerRoutes);
app.use("/api/compile/", cppCompilerRoutes);
app.use("/api/compile/", javaCompilerRoutes);
app.use("/api/compile/", pythonCompilerRoutes);
```

Snippet B: JWT authentication middleware

```
// Server/Middleware/authMiddleware.js
export const authenticateUser = (req, res, next) => {
```

```

let token = req.cookies.authToken || req.cookies.SuperauthToken;
if (!token) {
  const authHeader = req.header("Authorization");
  if (authHeader) {
    token = authHeader.startsWith("Bearer ")
      ? authHeader.slice(7)
      : authHeader;
  }
}
if (!token)
  return res
    .status(401)
    .json({ message: "Access denied! No token provided" });
try {
  const decoded = jwt.verify(token, process.env.JWT_SECRET);
  req.user = decoded;
  next();
} catch (error) {
  res.status(401).json({ message: "Invalid token" });
}
};

```

Snippet C: Note upload validation and creation

```

// Server/Controllers/noteController.js
if (!title || !description || !course || !semester || !session || !subject) {
  throw new AppError("All fields are required", 400);
}
if (!req.file) {
  throw new AppError("File is required", 400);
}
const admin = await Admin.findById(req.user.id).select("course department");
if ((course || "").toLowerCase() !== (admin.course || "").toLowerCase()) {
  throw new AppError("You can upload notes only for your assigned course", 403);
}

```

Snippet D: Notes search and filtering

```

// Server/Controllers/noteController.js
if (query && String(query).trim()) {
  filter.$text = { $search: String(query).trim() };
}
if (subject) filter.subject = { $regex: `^${subject}$`, $options: "i" };
if (course) filter.course = { $regex: `^${course}$`, $options: "i" };
if (semester) filter.semester = { $regex: `^${semester}$`, $options: "i" };
if (branch) filter.branch = { $regex: `^${branch}$`, $options: "i" };
if (session) filter.session = { $regex: `^${session}$`, $options: "i" };

```

Snippet E: PostgreSQL sandbox (transaction + rollback)

```
// Server/Compilers/PostGres/PostgresCompiler.js
await client.query("BEGIN");
await client.query("SET LOCAL statement_timeout = 10000");
await client.query(SANDBOX_BOOTSTRAP_SQL);
const rawResult = await client.query(query);
await client.query("ROLLBACK");
```

Snippet F: Axios with cookie-based auth

```
// Client/src/Api/axiosInstance.js
const API = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
  withCredentials: true,
  headers: { "Content-Type": "application/json" },
});
```

Snippet G: AuthContext session checks

```
// Client/src/Context/AuthContext.jsx
const saResponse = await API.get("/superadmin/profile", authProbeConfig);
const adminResponse = await API.get("/auth/admin/me", authProbeConfig);
const userResponse = await API.get("/auth/me", authProbeConfig);
```

Frontend/Backend Connectivity

- Vite proxy forwards /api to the backend in development.
- Axios uses withCredentials to attach cookies to API requests.
- Backend CORS allows credentials and a configured FRONTEND_URL.
- The app uses role-specific protected routes to guard UI access.

CHAPTER V: TESTING, RESULTS AND APPLICATION

Test Strategy

- Manual functional testing for core flows (auth, note upload, search, feedback).
- API endpoint checks using browser and tools such as cURL or API tester.
- Compiler modules tested with sample code snippets.
- SQL sandbox tested with sample SELECT queries on temp tables.

Test Case Table (Functional)

TC ID	Test Case	Input	Expected Output	Status
TC-01	User signup	name, email, password	Verification email sent	[PASS/FAIL]
TC-02	User login	valid credentials	authToken cookie set	[PASS/FAIL]
TC-03	Admin login	valid credentials	admin session active	[PASS/FAIL]
TC-04	Note upload	metadata + file	Note stored and Cloudinary URL	[PASS/FAIL]
TC-05	Notes search	subject + query	Filtered notes list	[PASS/FAIL]
TC-06	Feedback submit	message + rating	Feedback stored	[PASS/FAIL]
TC-07	Chatbot message	message	AI reply returned	[PASS/FAIL]
TC-08	SQL sandbox	SELECT query	Result rows and fields	[PASS/FAIL]
TC-09	C compiler	C program	stdout or error message	[PASS/FAIL]
TC-10	Python compiler	Python program	stdout or error message	[PASS/FAIL]

UI Screenshot Placeholders

- Figure 5.1: Home page (latest notes and stats)
- Figure 5.2: Login screen
- Figure 5.3: Dashboard and academic navigation
- Figure 5.4: Notes list with filters
- Figure 5.5: Admin upload notes form
- Figure 5.6: SuperAdmin academic management
- Figure 5.7: Tools hub
- Figure 5.8: Compiler editor page

Output Analysis (Representative Responses)

- `/api/notes/home-data` returns:
 - latestNotes array
 - stats object (students, notes, subjects, branches)
 - testimonials array (feedback data)
- `/api/notes/search` supports:
 - text search using MongoDB text index
 - query parameters for course, branch, semester, subject, session
 - pagination with page and limit
- `/api/query/sandbox` executes SQL within a transaction and returns rows and fields.

Real-world Applications

- Department-level notes repository for B.Tech programs.
- Internal academic tools hub for lab practice and programming exercises.
- Centralized portal for event announcements and feedback collection.

- Student assistant interface for FAQs and navigation guidance.

CHAPTER VI: ADVANTAGES, LIMITATIONS AND FUTURE SCOPE

Advantages

- Clear academic hierarchy and discoverability of notes.
- Multi-role access (student, admin, super admin) with secure auth.
- Cloudinary integration for file storage and updates.
- Built-in tools hub and compilers for practical learning.
- Dockerized setup for reproducible dev and prod environments.
- Extensible API design with modular controllers and routes.

Limitations (observed in repository)

- Automated tests are not defined in package scripts (server test script is placeholder).
- Some learning tools are documented and listed in the tools hub, but routes are not implemented in App.jsx.
- Rate limiter thresholds are currently configured with high limits (1000) for testing.
- Example environment file includes real-looking values; secure handling and sanitization are needed for production.

Future Scope (from project improvement documents)

- Add analytics dashboards for admins and super admins (active users, top subjects, uploads).
- Implement notifications for new notes and verification status.
- Introduce semantic search and ranking for notes.
- Add global request validation and audit logs.
- Add CI/CD, automated tests, and staged deployments.
- Expand tools hub with fully wired routes for regex tester, JSON formatter, DSA visualizer, markdown notes pad, API tester, Linux terminal simulator, ER diagram builder, MCQ practice engine, and progress tracker.

REFERENCES (author-year style)

- Project Team (2026). The College README.md. Repository documentation.
- Project Team (2026). Server Documentation (server-documentation.md, Doc.md).
- Project Team (2026). Docker Setup Guide (docs/DOCKER_SETUP.md).
- Project Team (2026). Tool Design Documents (docs/*.md).
- Project Team (2026). Component Refactoring Report (Related_Data_Testing/MDFiles/Component_Refactoring_Report.md).
- Project Team (2026). Copyright Application and Invention Disclosure (Related_Data_Testing/COPYRIGHT_APPLICATION.md,

Related_Data_Testing/MDFiles/INVENTION_DISCLOSURE/README.md).

TABLE INDEX

- Table 1.1: Existing vs Proposed System
 - Table 2.1: Functional Requirements
 - Table 2.2: Database Table Schema
 - Table 5.1: Functional Test Cases
-

FIGURE INDEX

- Figure 3.1: Overall System Architecture
 - Figure 3.2: DFD Level 0
 - Figure 3.3: DFD Level 1
 - Figure 3.4: ER Diagram
 - Figure 3.5: System Flowchart
 - Figure 5.1: Home Page Screenshot (placeholder)
 - Figure 5.2: Login Screen (placeholder)
 - Figure 5.3: Dashboard Navigation (placeholder)
 - Figure 5.4: Notes List (placeholder)
 - Figure 5.5: Admin Upload Notes (placeholder)
 - Figure 5.6: SuperAdmin Academic Management (placeholder)
 - Figure 5.7: Tools Hub (placeholder)
 - Figure 5.8: Compiler Editor (placeholder)
-

APPENDICES

Appendix A: Backend File Summary (Server)

Config

- Config/cloudinary.js: Cloudinary configuration using environment variables.
- Config/firebaseAdmin.js: Firebase Admin initialization for token verification.
- Config/FileAllowType.md: Documentation on allowed upload formats.

Database

- Database/db.js: MongoDB connection helper with env checks.

Controllers

- Controllers/authController.js: user registration, login, verification, profile updates, logout, admin user listing.

- Controllers/AdminController.js: admin registration, login, verification, profile updates.
- Controllers/SuperAdminController.js: super admin auth, profile, stats, user/admin management.
- Controllers/AcademicController.js: CRUD for sessions, courses, branches, semesters, subjects.
- Controllers/noteController.js: note upload, list, update, delete, search, public home data.
- Controllers/feedbackController.js: submit, list, delete feedback.
- Controllers/ContactController.js: submit, list, delete contact requests.
- Controllers/SubscribeController.js: subscribe, unsubscribe, list, delete.
- Controllers/chatbotController.js: AI assistant endpoint.
- Controllers/socialAuthController.js: Firebase social login.
- Controllers/EventPostController.js: empty file (placeholder).

Middleware

- Middleware/authMiddleware.js: JWT validation and admin role check.
- Middleware/SuperAdminMiddleware.js: SuperAdmin JWT validation.
- Middleware/uploadMiddleware.js: Cloudinary file upload for notes.
- Middleware/multerMiddlewareForProFilePic.js: Cloudinary upload for profile images.
- Middleware/RateLimiter.js: express-rate-limit for login and chatbot.
- Middleware/noteLimiter.js: rate limit skeleton for notes (not used).
- Middleware/authLimiter.js: rate limit skeleton for auth (not used).

Models

- Models/UserModel.js: user schema with role and authProvider.
- Models/AdminModel.js: admin schema with course, department, college, designation.
- Models/SuperAdminModel.js: super admin schema.
- Models/Note.js: notes with dynamic uploader and academic metadata.
- Models/SessionModel.js: academic session data.
- Models/CourseModel.js: course data.
- Models/BranchModel.js: branch data.
- Models/SemesterModel.js: semester data.
- Models/SubjectModel.js: subject data.
- Models/Feedback.js: feedback entries.
- Models/ContactUSmodel.js: contact form entries.
- Models/SubscribedEmailModel.js: newsletter subscriptions.
- Models/EventModel.js: event data.

Routes

- Routes/authRoutes.js: user/admin auth endpoints.
- Routes/AcademicRoutes.js: academic data endpoints.
- Routes/noteRoutes.js: note upload and search endpoints.
- Routes/feedbackRoutes.js: feedback endpoints.
- Routes/ContactRoute.js: contact endpoints.
- Routes/SubscribeRoute.js: subscribe endpoints.
- Routes/SuperAdminRoute.js: super admin endpoints.
- Routes/chatbotRoutes.js: chatbot endpoint.

Compilers

- Compilers/C_Programming/ServerForC.js: C compiler route using gcc.
- Compilers/Cpp_Programming/ServerForCpp.js: C++ compiler route using g++.
- Compilers/Java/ServerForJava.js: Java compile and run.
- Compilers/Python_Compiler/ServerForPython.js: Python run with fallback to py -3.
- Compilers/Postgres/PostgresCompiler.js: SQL query and sandbox endpoints.
- Compilers/Postgres/ConnectionPool.js: pg pool configuration.

Utils

- utils/AppError.js: custom error type.
- utils/catchAsync.js: async handler wrapper.
- utils/UserEmailVerification.js: user verification email.
- utils/AdminEmailVerification.js: admin verification email.
- utils/SuperAdminEmailVerification.js: super admin verification email.

Views

- views/UserEmailVerify.ejs, AdminEmailVerify.ejs, SuperAdminEmailVerify.ejs: verification views.

Scripts

- scripts/seedAcademicData.js, freshSeed.js, dedupAcademicData.js, seedAcademicDataByGPT.js: academic data seed utilities.
- scripts/check_admins.js, testGeminiKey.js: admin and chatbot helper scripts.

Appendix B: Frontend File Summary (Client)

Root

- index.html: client entry HTML, title "Home | Hellomates".
- vite.config.js: Vite config with /api proxy and COOP headers for Firebase popup.
- tailwind.config.js: Tailwind content paths and theme extensions.

Entry and Core

- src/main.jsx: React root with AuthProvider.
- src/App.jsx: Router configuration for public and protected pages.
- src/Api/axiosInstance.js: Axios configuration with cookies and auth error handling.
- src/Context/AuthContext.jsx: user/admin/superAdmin session management.

Protected Routes

- src/Config/ProtectedUserRoute.jsx: user route guard.
- src/Config/ProtectedAdminRoute.jsx: admin route guard.
- src/Config/ProtectedSuperAdminRoute.jsx: super admin route guard.

Firebase

- src/Config/firebase.js: Firebase client config and providers.

Primary Pages (routes)

- Pages/Home.jsx: home UI with latest notes, stats, testimonials.
- Pages/Login.jsx, Pages/SignUp.jsx: user auth UI.
- Pages/AdminPages/AdminLogin.jsx, AdminSignUp.jsx, AdminDashboard.jsx, ManageNotes.jsx, AllUsers.jsx, AdminProfile.jsx, NoteUpload/UploadNote.jsx: admin flows.
- Pages/SuperAdminPages/SuperAdminLogin.jsx, SuperAdminDashboard.jsx, AcademicManagement.jsx: super admin flows.
- Pages/Assistant.jsx: AI assistant chat UI.
- Pages/Contact.jsx, Feedback.jsx, Events.jsx, Terms.jsx, Privacy.jsx, Support.jsx, About.jsx, Services.jsx: informational and feedback pages.

Academic Navigation Components

- Sessions/Dashboard.jsx: sessions listing.
- Courses/Courses.jsx and components: course list and details.
- Branches/Branches.jsx and components: branch list and details.
- Semesters/Semester.jsx and components: semester list and details.
- Subject/Subjects.jsx and components: subject list and details.
- Notes/NotesList.jsx and components: notes list, filtering, and cards.

Tools Hub and Compilers

- Pages/Compilers/CompilersHome.jsx: tools catalog and navigation links.
- Pages/Compilers/PostgressCompiler/PostgresEditor.jsx: SQL editor UI.
- Pages/Compilers/JSCompilers/JsEditor.jsx: JS editor UI.
- Pages/Compilers/C_programming/CEditor.jsx: C editor UI.
- Pages/Compilers/Cpp_Compiler/CppEditor.jsx: C++ editor UI.
- Pages/Compilers/Java_Compiler/JavaEditor.jsx: Java editor UI.
- Pages/Compilers/Python_Compiler/PythonEditor.jsx: Python editor UI.
- Pages/Compilers/HTML_CSS_Playground/HtmlCssPlayground.jsx: HTML/CSS playground UI.
- Pages/Tools/Basic Learning Tools/Git_Practice_Lab/GitPracticeLab.jsx: Git practice lab UI.

Components

- Components/Navbaar/*: Navbar and subcomponents.
- Components/Footer.jsx: site footer.
- Components/NotFound.jsx: 404 page.
- Components/Breadcrumb.jsx, Slider.jsx, Carousel.jsx, CallToAction.jsx: UI sections.

Appendix C: Documentation Inventory (docs)

- API_TESTER.md: design for browser-based API tester tool.
- C_COMPILER.md, CPP_COMPILER.md, JAVA_COMPILER.md, PYTHON_COMPILER.md: compiler tool designs.
- JAVASCRIPT_RUNTIME.md: JS runtime tool design.

- ## Appendix D: Related_Data_Testing (Prototype Area)

- ## Appendix E: Glossary and Abbreviations

- ## Appendix F: Additional Test Cases (Extended)

19 / 20

subjects | | TC-14 | Subscribe | POST /api/subscribe | email | Subscription success | | TC-15 | Unsubscribe | POST /api/subscribe/unsubscribe | email | Unsubscribe success |

END OF REPORT